

# Taller — Sistema Bancario

Herencia · super · Sobrecarga · Sobreescritura · Polimorfismo

Java · MJ4 — se apoya en MJ2 (clases y encapsulación) y MJ1 (métodos)

---

## Contexto

El Banco Nacional Andino necesita un sistema que gestione diferentes tipos de cuentas bancarias. Todas las cuentas comparten información básica —número de cuenta, titular y saldo actual— pero cada tipo tiene reglas propias para cobrar comisiones, calcular intereses y describirse al cliente.

El banco ofrece tres productos:

- **Cuenta de Ahorros.** Genera intereses mensuales según la tasa vigente. No cobra comisión por retiros normales, pero sí por retiros que dejen el saldo por debajo del mínimo permitido.
- **Cuenta Corriente.** Diseñada para empresas. Cobra una comisión fija por cada transacción. Permite saldo en negativo hasta un límite de sobregiro autorizado.
- **Cuenta de Inversión.** Bloquea el dinero por un plazo en meses. Ofrece la tasa más alta, pero cobra una penalización si el cliente retira antes del plazo.

*Tu trabajo es modelar este sistema en Java aplicando los conceptos vistos en clase.*

## Recuerda

### Herencia

Una clase hija hereda todos los miembros no privados de su clase padre. Se declara con `extends`. Establece una relación es-un.

### `super()`

Llama al constructor del padre. Debe ser la primera línea del constructor de la hija. Permite inicializar los campos del padre a través de él.

### Sobrecarga

Mismo nombre de método, distinta firma (número o tipo de parámetros). El compilador decide cuál invocar antes de ejecutar el programa.

### Sobreescritura (`@Override`)

La hija reemplaza un método del padre con su propia versión, usando la misma firma exacta. Java decide cuál ejecutar en tiempo de ejecución, según el tipo real del objeto.

### `super.metodo()`

Llama la versión del método definida en el padre, desde dentro de la hija. Útil para extender el comportamiento sin borrarlo.

### Polimorfismo

Una variable del tipo padre puede guardar instancias de cualquier hija. Al llamar un método sobreescrito, Java ejecuta la versión del objeto real, no la del tipo declarado de la variable.

## Tareas

### Tarea 1 — Clase padre: CuentaBancaria

- Crea la clase `CuentaBancaria` con tres campos **privados**: `numeroCuenta` (String), `titular` (String) y `saldo` (double).
- Escribe un constructor que reciba los tres campos y los inicialice usando `this`.
- Agrega un getter para cada campo. El saldo, además, debe tener un método **protegido** `setSaldo(double nuevoSaldo)` — solo las clases hijas podrán usarlo.
- Agrega `String describir()` que retorne: `"Cuenta [numeroCuenta] · Titular: [titular] · Saldo: $[saldo]"`
- Agrega `double calcularComision()` que retorne `0.0` — es la comisión base; cada hija la ajustará.
- Agrega `void realizarRetiro(double monto)` que descuente el monto del saldo usando `setSaldo`. Por ahora no valida nada.

#### Por qué `protected` en `setSaldo`

El saldo es privado: nadie de afuera lo cambia a la fuerza. Pero las hijas necesitan ajustarlo con sus propias reglas. `protected` es exactamente ese permiso: acceso para la clase y sus hijas, no para el resto. No instancias esta clase directamente en Main — representa el contrato común de todas las cuentas.

### Tarea 2 — Clase hija: CuentaAhorros

- Crea `CuentaAhorros` que extienda `CuentaBancaria`.
- Agrega dos campos propios: `tasaInteresMensual` (double) y `saldoMinimo` (double).
- Constructor con `numeroCuenta`, `titular`, `saldo`, `tasaInteresMensual`, `saldoMinimo`. Usa `super()` para los tres primeros.
- Sobreescribe `describir()`: llama `super.describir()` y agrega `" | Tasa mensual: [tasaInteresMensual]%"`.
- Sobreescribe `calcularComision()`: retorna `0.0` si el saldo es mayor o igual al mínimo, o `12000.0` si está por debajo.
- Sobrecarga `realizarRetiro`: agrega `realizarRetiro(double monto, boolean esUrgente)`.

#### Aclaración importante — evalúa el saldo resultante

La regla del retiro urgente es: si es urgente Y el saldo que quedaría después del retiro (`saldo - monto`) cae por debajo del mínimo, descuenta el monto y además la comisión fija de \$12000. Ojo: debes evaluar el saldo **RESULTANTE**, no el actual. Si preguntas por `calcularComision()` antes de retirar, mirará el saldo actual (que todavía puede estar sobre el mínimo) y no reflejará el retiro. Calcula primero `saldo - monto` y decide sobre ese valor.

#### Ejemplo numérico

Saldo \$600000, mínimo \$500000. Retiro urgente de \$200000 → el resultante sería \$400000 (bajo el mínimo) → se cobra la comisión:  $600000 - 200000 - 12000 = \$388000$ .

- Agrega `double calcularInteresDelMes()` que retorne `getSaldo() * tasaInteresMensual / 100`.

### Tarea 3 — Clase hija: CuentaCorriente

- Crea CuentaCorriente que extienda CuentaBancaria.
- Campos propios: `comisionPorTransaccion` (double) y `limiteSobregiro` (double) — cuánto puede quedar el saldo en negativo.
- Constructor con `numeroCuenta`, `titular`, `saldo`, `comisionPorTransaccion`, `limiteSobregiro`. Usa `super()`.
- Sobreescribe `describir()`: usa `super.describir()` y agrega " | Comisión por transacción: \$ [comisionPorTransaccion]".
- Sobreescribe `calcularComision()`: retorna la `comisionPorTransaccion`.
- Sobreescribe `realizarRetiro(double monto)`: calcula el saldo resultante como `saldo - monto - comisionPorTransaccion`. Si ese resultado queda por debajo de `-limiteSobregiro`, imprime que el retiro no es posible y no cambia el saldo. Si es válido, aplica el descuento.

#### Aclaración sobre el signo del límite

El límite de sobregiro es un número positivo (ej. \$300000) que representa qué tan negativo puede llegar el saldo. Es decir: el saldo nunca debe bajar de `-limiteSobregiro`. La condición de rechazo es `saldoResultante < -limiteSobregiro`. Piensa en `-limiteSobregiro` como el piso del saldo.

#### Ejemplo numérico

Saldo \$100000, comisión \$5000, límite de sobregiro \$300000 (piso: `-$300000`). Retiro de \$200000 → `100000 - 200000 - 5000 = -$105000`. Como `-105000` no baja de `-300000`, se permite. Un retiro adicional de \$300000 daría `-$410000`, que sí baja del piso → se rechaza y el saldo no cambia.

### Tarea 4 — Clase hija: CuentaInversion

- Crea CuentaInversion que extienda CuentaBancaria.
- Campos propios: `tasaAnual` (double), `plazoMeses` (int) y `penalizacionRetiroAnticipado` (double).
- Constructor con todos los campos necesarios. Usa `super()` para los tres del padre.
- Sobreescribe `describir()`: usa `super.describir()` y agrega " | Plazo: [plazoMeses] meses · Tasa anual: [tasaAnual] %".
- Sobreescribe `calcularComision()`: retorna la `penalizacionRetiroAnticipado`.
- Sobreescribe `realizarRetiro(double monto)`: siempre descuenta la penalización además del monto: `saldo - monto - penalizacion`.
- Sobrecarga `calcularComision`: agrega `calcularComision(int mesesTranscurridos)` que retorne `0.0` si ya se cumplió el plazo (`mesesTranscurridos >= plazoMeses`), o la penalización normal si aún no.

#### Aquí ves sobrecarga y sobreescritura juntas

`calcularComision()` (sin parámetros) sobreESCRIBE la del padre. `calcularComision(int)` es una sobreCARGA: mismo nombre, firma distinta. Es el contraste exacto que vimos en clase — obsérvalo en una misma clase.

## Tarea 5 — Polimorfismo en acción: Main

- Crea tres variables de tipo `CuentaBancaria` y asígnales una `CuentaAhorros`, una `CuentaCorriente` y una `CuentaInversion`.
- Llama `describir()` sobre cada variable e imprime el resultado. Observa que Java ejecuta la versión correcta para cada tipo.
- Llama `calcularComision()` sobre cada variable. Sin saber el tipo real, el sistema calcula la comisión correcta.
- Llama `realizarRetiro(500000.0)` sobre cada cuenta e imprime el saldo con `getSaldo()`. Observa el comportamiento distinto de cada una.
- Responde en un comentario dentro del código: ¿por qué fue necesario usar `CuentaBancaria` como tipo de variable en lugar de cada tipo específico? ¿Qué ventaja concreta te dio?

### Este es el núcleo del taller

Si el polimorfismo está bien implementado, Main no necesita saber qué tipo de cuenta tiene cada variable para que todo funcione. Ese es el punto.

## Salida esperada (para autoverificarte)

Con estos datos de ejemplo en tu Main:

```
CuentaBancaria c1 = new CuentaAhorros("AH-001", "Ana Gómez", 1000000.0, 0.5, 500000.0);
CuentaBancaria c2 = new CuentaCorriente("CC-002", "Distribuidora XYZ", 800000.0, 5000.0,
300000.0);
CuentaBancaria c3 = new CuentaInversion("IN-003", "Carlos Ruiz", 2000000.0, 8.0, 12,
50000.0);
```

Al ejecutar `describir()`, `calcularComision()` y `realizarRetiro(500000.0)` sobre cada una, deberías ver:

```
Cuenta AH-001 · Titular: Ana Gómez · Saldo: $1000000.0 | Tasa mensual: 0.5%
Cuenta CC-002 · Titular: Distribuidora XYZ · Saldo: $800000.0 | Comisión...: $5000.0
Cuenta IN-003 · Titular: Carlos Ruiz · Saldo: $2000000.0 | Plazo: 12 meses · Tasa anual:
8.0%
----
Comisión c1: $0.0      (saldo 1000000 >= mínimo 500000)
Comisión c2: $5000.0   (comisión por transacción)
Comisión c3: $50000.0  (penalización)
----
Saldo c1: $500000.0    (1000000 - 500000, retiro normal)
Saldo c2: $295000.0    (800000 - 500000 - 5000 comisión)
Saldo c3: $1450000.0   (2000000 - 500000 - 50000 penalización)
```

### Nota sobre el formato del saldo

Verás el saldo como \$1000000.0 (con .0 y sin puntos de miles). Es normal: un double en Java se imprime así. Dar formato bonito a la moneda no es parte de este taller — no te preocupes por eso ahora.

## Para reflexionar

Cuando termines las cinco tareas, piensa en esta situación: el banco decide agregar un cuarto tipo de cuenta — `CuentaNomina` — con sus propias reglas de comisión y retiro.

¿Tendrías que modificar el código de `Main` para que funcione con las variables que ya declaraste? ¿Por qué?

#### Pista para pensar

Si `Main` solo conoce el tipo `CuentaBancaria` y llama métodos que existen en el padre, ¿qué tendría que cambiar cuando aparece una hija nueva? Esa respuesta es una de las mayores ventajas de la herencia y el polimorfismo bien usados.